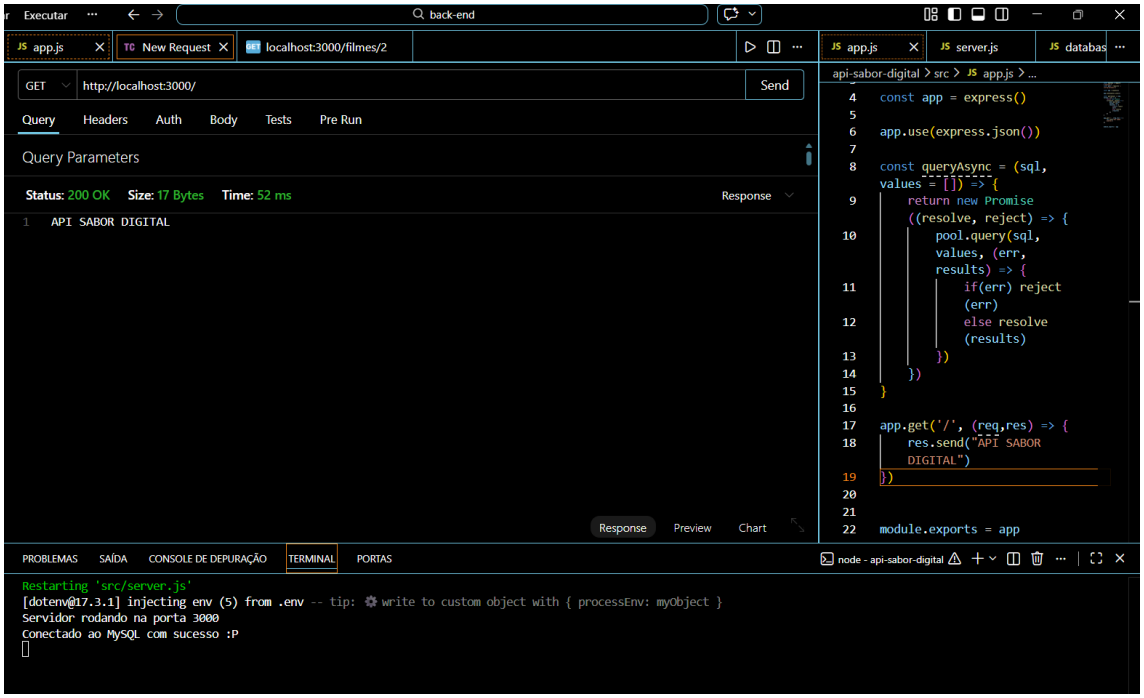
Atividade Formativa

Lívia Hermano n°19

- O Node.js com Express é adequado para esta API REST devido o seu uso ser unificado com o JavaScript, assim tendo mais simplicidade e facilidade em sua programação, além de garantir maior alta performance.



-Teste do GET inicial

```
GET  http://localhost:3000/produtos                Send
Status: 200 OK   Size: 440 Bytes   Time: 41 ms      Response
{
  "dados": [
    {
      "id": 3,
      "nome": "Nhoque ao Pesto",
      "descricao": "Feito com molho especial",
      "preco": "28.21",
      "disponivel": 1
    },
    {
      "id": 2,
      "nome": "Brownie com Sorvete",
      "descricao": "Uma explosão de sabores, totalmente
          irresistível",
      "preco": "17.88",
      "disponivel": 1
    },
    {
      "id": 1,
      "nome": "Espaguete à Carbonara",
      "descricao": "O melhor espaguete que você irá comer, com
          massa feita diretamente da Itália",
      "preco": "23.89",
```

```
app.get('/', (req,res) => {
    res.send("API SABOR DIGITAL")
})

app.get('/produtos', async (req,res) => {
    try{
        const produtos = await queryAsync ('SELECT * FROM produto
        ORDER BY id DESC')
        res.json({
            sucesso: true,
            dados: produtos,
            total: produtos.length
        })
    } catch (erro) {
        console.error('Erro ao listar produtos: ', erro)
        res.status(500).json({
            sucesso: false,
            mensagem: 'Erro ao listar produtos',
            erro: erro.message
        })
    }
})
```

```
Restarting 'src/server.js'
[dotenv@17.3.1] injecting env (5) from .env -- tip: write to custom object with { processEnv: myObject }
Servidor rodando na porta 3000
Conectado ao MySQL com sucesso :P
```

- Teste do GET para listar os produtos

```
GET  http://localhost:3000/produtos/2              Send
Query  Headers  Auth  Body  Tests  Pre Run
Status: 200 OK   Size: 158 Bytes   Time: 14 ms      Response
{
  "sucesso": true,
  "dados": {
    "id": 2,
    "nome": "Brownie com Sorvete",
    "descricao": "Uma explosão de sabores, totalmente irresistível"
        ,
    "preco": "17.88",
    "disponivel": 1
  }
}
```

```
app.get('/produtos/:id', async (req,res) => {
        }
        const produto = await queryAsync ('SELECT * FROM produto
        WHERE id = ?', [id])
            if(produto.length === 0){
                return res.status(404).json({
                    sucesso: false,
                    mensagem: 'Produto não encontrado'
                })
            }
            res.json({
                sucesso: true,
                dados: produto [0]
            })
        } catch (erro) {
        console.error('Erro ao listar produtos: ', erro)
        res.status(500).json({
            sucesso: false,
            mensagem: 'Erro ao listar produtos',
            erro: erro.message
        })
    }
})

module.exports = app
```

```
Restarting 'src/server.js'
[dotenv@17.3.1] injecting env (5) from .env -- tip: secrets for agents: https://dotenvx.com/as2
Servidor rodando na porta 3000
Conectado ao MySQL com sucesso :P
```

-Teste do GET de produto específico.

```
POST http://localhost:3000/produtos  Send
Query  Headers  Auth  Body  Tests  Pre Run
JSON  XML  Text  Form  Form-encode  GraphQL  Binary
{
  "nome": "Torta de limão",
  "descricao": "A mais deliciosa torta de limão, com receita da casa",
  "preco": 12.50,
  "disponivel": true
}
Status: 201 Created  Size: 51 Bytes  Time: 26 ms  Response
{
  "sucesso": true,
  "mensagem": "Produto criado",
  "id": 4
}
Response  Chart
```

```
app.js M  server.js  database.js  .env
api-sabor-digital > src > app.js > ...
70  app.post('/produtos', async (req,res) =>{
71      try{
72          const {nome, descricao, preco, disponivel} = req.body
73
74          if(!nome || !descricao || !preco || !disponivel){
75              return res.status(400).json({
76                  sucesso: false,
77                  mensagem: "Nome, descrição, preço e disponibilidade são itens obrigatórios"
78              })
79          }
80          if(typeof preco !== 'number' || preco <=0){
81              return res.status(400).json({
82                  sucesso: false,
83                  mensagem: "Preço tem que ser um valor positivo"
84              })
85          }
86          const novoProduto = {
87              nome: nome.trim(),
88              descricao: descricao.trim(),
89              preco: preco,
90              disponivel: disponivel
91          }
92          const resultado = await queryAsync('INSERT INTO produto SET ?', [novoProduto])
93
94          res.status(201).json({
```

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
node - api-sabor-digital
Restarting 'src/server.js'
[dotenv@17.3.1] injecting env (5) from .env -- tip: agentic secret storage: https://dotenvx.com/as2
Servidor rodando na porta 3000
Conectado ao MySQL com sucesso :P
```

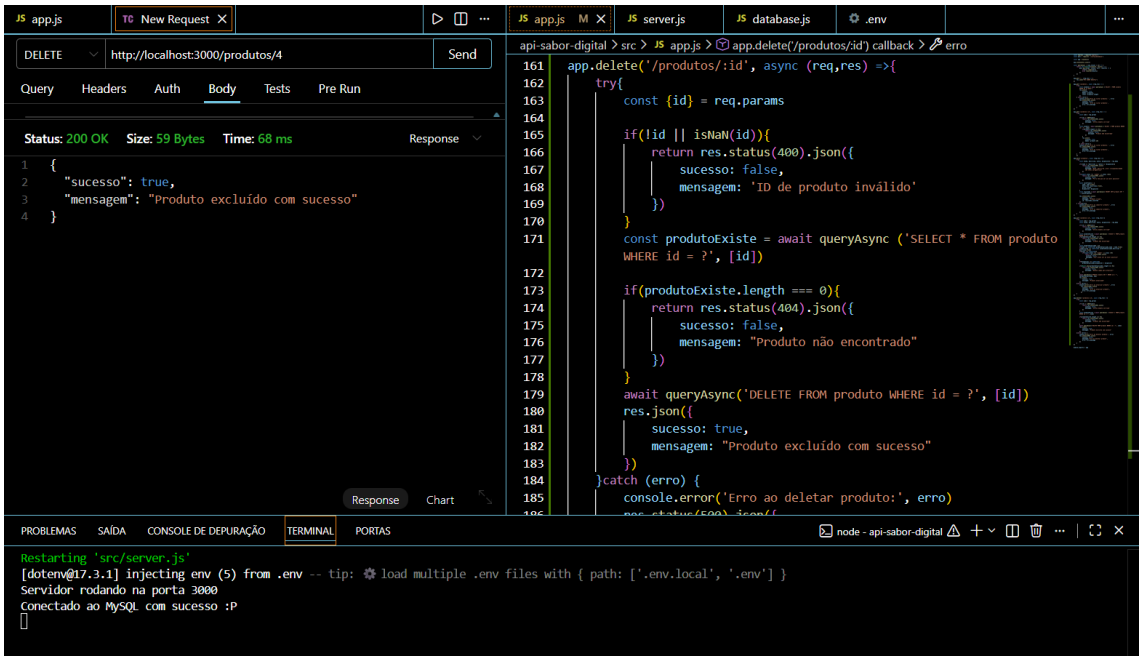
-Teste do POST para cadastrar novos produtos.

```
PUT http://localhost:3000/produtos/4  Send
Query  Headers  Auth  Body  Tests  Pre Run
JSON  XML  Text  Form  Form-encode  GraphQL  Binary
{
  "nome": "Torta de limão",
  "descricao": "A mais deliciosa torta de limão, com receita da casa",
  "preco": 15.50,
  "disponivel": true
}
Status: 200 OK  Size: 48 Bytes  Time: 46 ms  Response
{
  "sucesso": true,
  "mensagem": "Produto atualizado"
}
Response  Chart
```

```
app.js M  server.js  database.js  .env
api-sabor-digital > src > app.js > ...
109  app.put('/produtos/:id', async (req,res)=>{
110      try{
111          const {id} = req.params
112          const {nome, descricao, preco, disponivel} = req.body
113
114          if(!id || isNaN(id)){
115              return res.status(400).json({
116                  sucesso: false,
117                  mensagem: "ID de produto inválido"
118              })
119          }
120          const produtoExiste = await queryAsync ('SELECT * FROM produto WHERE id = ?', [id])
121          if(produtoExiste.length === 0){
122              return res.status(404).json({
123                  sucesso: false,
124                  mensagem: 'Produto não encontrado'
125              })
126          }
127          const produtoAtualizado = {}
128          if(nome !== undefined) produtoAtualizado.nome = nome.trim()
129          if(descricao !== undefined) produtoAtualizado.descricao = descricao.trim()
130          if(preco !== undefined){
131              if(typeof preco !== 'number' || preco <=0)
132                  return res.status(400).json({
133                      sucesso: false,
```

```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
node - api-sabor-digital
Restarting 'src/server.js'
[dotenv@17.3.1] injecting env (5) from .env -- tip: secrets for agents: https://dotenvx.com/as2
Servidor rodando na porta 3000
Conectado ao MySQL com sucesso :P
```

- Teste do PUT para atualizar produto específico.



- Teste do DELETE de produtos específicos.